

Testeo de funciones sin Jasmine

1. Función comprobar si un número es par	2
2. Función calificación con nota entera	3
3. Función calificación con nota decimal	3
4. Calcular perímetro y área de una circunferencia	4
5. Función comprobar si un año es bisiesto	4
6. Conversor de hexadecimal a decimal	5

1. Función comprobar si un número es par

Crear un documento HTML con nombre: `comprobar_paridad.html`

Para la función creada anteriormente `comprobarEsPar()` :

Prepara una función de testeo llamada `test1()` que llame a la función `comprobarEsPar` pasándole como argumento de entrada números en el rango de 0 a 100, y que vuelque por la consola la salida de la función. Puedes añadir otro botón a tu página web de manera que cuando el usuario lo pulse llame a la función testear.

Prepara los siguientes arrays con valores de entrada y de valores esperados:

```
let valoresEntrada=[1,2,5,8,9,12,16];  
let valoresEsperados=[false,true,false,true,false,true,true];
```

En vez de los dos arrays anteriores se podría crear un array de objetos que encapsule tanto los valores de entrada como las salidas esperadas:

```
let datos = [  
  {entrada:1, valorEsperado: false},  
  {entrada:2, valorEsperado: true},  
  {entrada:5, valorEsperado: false},  
  {entrada:8, valorEsperado: true},  
  {entrada:9, valorEsperado: false},  
  {entrada:12, valorEsperado: true},  
  {entrada:16, valorEsperado: true}  
];
```

Prepara una segunda función de testeo llamada `test2()` que se encargue de ir llamando a la función a testear con los valores de entrada y vaya comprobando si la función devuelve realmente el valor esperado. Se considerará que nuestra función ha superado el test si no ha fallado en ninguna de las comprobaciones anteriores.

2. Función calificación con nota entera

Crear un documento HTML con nombre: `calificacion_nota_entera.html`

Para la función creada anteriormente `verCalificacion()` :

Crea una **función de testeo** llamada `test()` que invoque a la función `verCalificacion` con un rango de números enteros comprendido entre -1 y 11 y que vuelque la salida de la función por la consola.

3. Función calificación con nota decimal

Crear un documento HTML con nombre: `calificacion_nota_decimal.html`

La función creada anteriormente `verCalificacionDecimal()` :

Crea un array con los siguientes valores de entrada:

```
let notas =  
[-1.5, 0, 1, 2, 3, 4.5, 4.9, 5, 5.1, 5.9, 6, 6.1, 6.9, 7, 7.1, 7.9, 8, 8.1, 8.9, 9, 9.1, 9.9, 10, 11  
];
```

Crea otro array con los valores de salida esperados a cada uno de los anteriores valores de entrada (por ejemplo, para -1.5 el valor esperado es “VALOR INCORRECTO”, para 0 es “INSUFICIENTE”, etc.)

(Observación: también puedes crear un solo array que contenga objetos literales con las entradas y salidas como en el ejercicio anterior)

Crea una función de testeo llamada `test()` haga un testeo automático de la función `verCalificacionDecimal` con los valores de entrada y los valores esperados de los arrays creados anteriormente.

4. Calcular perímetro y área de una circunferencia

Crear un documento HTML con nombre: `calcula_circunferencia.html`

Para la función creada anteriormente `parametrosCircunferencia()` :

Debes escribir el código necesario para testear usando los datos de al menos tres circunferencias.

5. Función comprobar si un año es bisiesto

Crear un documento HTML con nombre: `comprobar_biesto.html`

Para la función creada anteriormente `esBisiesto()`:

Crea los siguientes las siguientes pruebas:

- Prepara un testeo que permita invocar a la función bisiesto con todos los años del siglo XX y XXI y vuelque a consola sólo aquellos que sí son bisiestos. Posteriormente compara la salida de consola con el listado de años bisiesto de la página web mencionada anteriormente ([Listado años bisiestos](#))
- Prepara una estructura capaz de contener los siguiente años bisiestos: 1600, 2000, 2016, 2020, 2400 y los siguientes años que no lo son: 1700, 1800, 1900, 2015 y 2019. Debes implementar un testeo automático que invoque a la función con esos datos de entrada y compruebe si la función ha devuelto los resultados esperados.

6. Conversor de hexadecimal a decimal

Crear un documento HTML con nombre: **conversor_hexadecimal_a_decimal.html**

Para las funciones creadas anteriormente **hexa2decimal()** y **digitoHexa2Dec()**:

Realiza un testeo automático con al menos 6 valores de entrada y valores esperados (recuerda que para las cifras hexadecimales debes usar tanto mayúsculas como minúsculas).

Para las comprobaciones puedes usar la calculadora de Windows en modo Programador o páginas web de conversión de sistemas numéricos online.